OMI-Port Canonical Authority

Universal Declarative Port Interface for Non-Connecting Source–Target Transforms

Status: Canonical Draft
Layer: Port Notation / Adapter Boundary / Declarative Interface
Authority: Canonical for port declaration shape only
Non-Authority: Does not connect, mount, transmit, validate, accept, receipt, or project
Depends on: OMI-IMO notation, FS/GS/RS/US pre-language scope topology, F* Gauge Family, OMI-Lisp declaration surface, OMI adapter boundary

---

0. Canonical Thesis

OMI-Port is a standardized declarative port interface for deriving pseudo-persistent OMI-IMO place-value routes from source and target scopes.

It is not a network stack.

It is not a filesystem.

It is not an operating system.

It is not POSIX.

It is not OSI runtime implementation.

It is a platform-agnostic declarative boundary that presents a common source-target interface before any transport authority exists.

Canonical rule:

```
OMI-Port declares a route.
OMI-Port does not connect the route.
```

---

1. Purpose

OMI-Port exists to standardize the shape of a dormant port.

A dormant port is a typed source-target opening that can be inspected, transformed, normalized, and handed downstream without becoming an active stream.

OMI-Port provides:

```
source declaration
target declaration
gauge selection
scope interpolation
route derivation
candidate handoff
adapter boundary
```

OMI-Port does not provide:

```
filesystem access
network access
socket access
hardware access
process execution
message delivery
validation
receipt
state acceptance
projection authority
```

One-line canon:

```
OMI-Port is a universal declarative adapter surface for deriving OMI-IMO place-
value route candidates without connecting them.
```

---

2. Why This Belongs Between Notation and Runtime

The OMI stack must separate four different things:

```
notation
    how a route is written

port declaration
    how source and target are named
```

```
adapter shape
    how the route is handed to downstream systems

runtime connection
    how an authorized implementation actually moves data
```

OMI-Port owns only the second layer.

It can define a standard declarative surface similar in spirit to a platform-agnostic “BusyBox of port declarations,” but it must not become a universal runtime.

BusyBox unifies many POSIX tools as executable commands.

OMI-Port unifies many source-target surfaces as dormant declarations.

Therefore:

```
BusyBox executes.
OMI-Port declares.

POSIX connects to operating behavior.
OMI-Port stops before operating behavior.
```

---

3. Canonical Scope Decision

The standardized part should be:

```
port notation
source-target grammar
scope representation
gauge transform declaration
candidate handoff shape
non-authority flags
```

The project-dependent part should remain:

```
actual filesystems
actual sockets
actual transports
actual DOM access
actual canvas access
```

```
actual LoRa access
actual serial access
actual storage
actual runtime permissions
actual validation
actual receipts
```

Thus OMI-Port MAY define a universal OSI-like adapter interface, but only as a non-connecting declarative surface.

Canonical rule:

```
Standardize the interface.
Do not standardize false authority.
```

---

4. OMI-Port Is Not OSI, But It Can Face OSI

OMI-Port may present a standardized interface for OSI-like concerns:

```
physical carrier reference
link reference
network reference
transport reference
session reference
presentation reference
application reference
```

But these are declarations only.

They are not active layers.

OMI-Port does not open a link, allocate a socket, mount a filesystem, negotiate a session, or transmit an application payload.

It may describe those layers as candidate scopes.

Example:

```
lora://915/node/alpha#/16
serial://ttyUSB0#/8
file:///repo/docs/spec.md#/64
```

```
https://example.com/api/v1#/40
canvas://board/card/7#/24
dom://document/main/button#/32
```

These strings name possible scopes.

They do not open them.

Canonical rule:

```
URI names.
CIDR scopes.
OMI routes.
Validation accepts or rejects.
Runtime connects only after authority.
```

---

5. URI-CIDR as Default Printable Port Scope

OMI-Port SHOULD use URI-CIDR as its default printable scope grammar:

```
<uri>#/<prefix-length>
```

Examples:

```
file:///repo/docs/spec.md#/64
https://example.com/api/v1#/40
dom://document/main/button#/32
lora://915/node/alpha#/16
serial://ttyUSB0#/8
canvas://board/card/7#/24
omi://scope/fs/gs/rs/us#/32
```

The URI identifies a carrier-facing namespace.

The CIDR suffix declares scope width.

The OMI transform derives the route.

The authority system later validates or rejects the route.

URI-CIDR is therefore a carrier grammar, not authority.

Canonical invariant:

```
URI-CIDR scopes.
It does not accept.
```

---

6. Canonical Transform Model

OMI-Port derives route notation through a gauge-selected transform:

```
PortTensor_G(source, target) → OMI-IMO route candidate
```

Expanded:

```
source    = declared starting scope
target    = declared stopping scope
G         = F* gauge/operator dialect
output    = OMI-IMO place-value route candidate
```

Canonical form:

```
T_G(source, target) = omi---imo
```

Expanded route surface:

```
o---o/---/?---?@---@
```

Machine-expanded route shape:

```
o-FS-GS-RS-US/FS/GS/RS/US?REGISTER?STACK@CAR@CDR
```

Meaning:

```
OMI side     = source-derived FS/GS/RS/US scope
IMO side     = target-derived FS/GS/RS/US scope
? fields     = witness / register / stack / query surface
@ fields     = CAR/CDR relation surface
```

Canonical rule:

```
The port tensor derives notation.
It does not connect the ports.
```

---

7. FS/GS/RS/US Is the Canonical Output Shape

OMI-Port MUST NOT use deprecated LL/MM/NN as its public canonical port notation.

The canonical output shape is FS/GS/RS/US.

Deprecated model:

```
LL/MM/NN
three-slot transition/lens model
historically useful
not canonical authority
```

Canonical model:

```
FS/GS/RS/US
four-scope memory topology
nibble-aligned
pre-language scope first
canonical for port output
```

Therefore:

```
LL/MM/NN remains a historical transform mnemonic.
FS/GS/RS/US defines canonical port topology.
```

---

8. F* Gauge Family as Dynamic Transform Dialect

OMI-Port uses the F* Gauge Family as the dynamic interpolation dialect.

Gauge pre-header:

```
G 00 1C 1D 1E 1F 20 G
```

with:

```
G ∈ 0xF0..0xFF
```

Interpretation:

```
G    selected gauge/operator dialect
00   NUL origin
1C   FS
1D   GS
1E   RS
1F   US
20   SP readable boundary
G    gauge closure
```

The F* gauge family gives dynamic transform selection around the fixed canonical FS/GS/RS/US spine.

Example transform readings:

```
F8   intersection-like / AND-like route
FE   union-like / OR-like route
F6   difference-like / XOR-like route
F9   equivalence-like / XNOR-like route
FB   implication-like source-to-target route
FD   converse implication-like target-to-source route
FF   tautological canonical closure
```

Canonical rule:

```
F* selects the transform dialect.
FS/GS/RS/US remains the canonical topology.
```

---

9. Dynamic Chirality and Byte-Level Port Creation

ASCII `O/o` supplies a portable chirality bit:

```
'O' = 0x4F
'o' = 0x6F
0x6F - 0x4F = 0x20
```

Resolver operations:

```
chirality_bit = byte & 0x20;
flipped       = byte ^ 0x20;
```

This generalizes beyond `O/o` to any ASCII carrier pair whose case relation toggles bit `0x20`.

Examples:

```
A ↔ a
B ↔ b
X ↔ x
O ↔ o
```

This is the historical insight preserved from the older dynamic LL/MM/NN-style gauge thinking: selected bytes can behave as sign/lens/operator carriers.

But canonical OMI-Port does not expose arbitrary byte flipping as authority.

Canonical rule:

```
Byte chirality may guide transform dialect.
It does not validate the route.
```

---

10. Folded and Unfolded Port Notation

OMI-Port recognizes two forms of the same route:

Folded compact form:

```
omi---imo
```

Unfolded expanded form:

```
o---o/---/?---?@---@
```

These are not the 3D/4D distinction.

They are folded and unfolded renderings of the same canonical OMI-IMO route.

Correct distinction:

```
LL/MM/NN      = deprecated three-slot transition model
FS/GS/RS/US   = canonical four-scope memory topology
```

Thus:

```
omi---imo
    folded route name

o---o/---/?---?@---@
    unfolded separator-rendered route surface

FS/GS/RS/US
    canonical machine topology
```

Canonical rule:

```
Folded and unfolded forms are route renditions.
FS/GS/RS/US is the canonical output topology.
```

---

11. Port Object Model

OMI-Port v0 defines these conceptual objects:

```
PortScope
    one declared URI-CIDR or OMI scope

PortTensor
    dynamic source-target interpolation rule selected by gauge

PortRoute
    derived OMI-IMO place-value route candidate
```

```
PortBinding
    declared candidate binding between source and target

PortPipe
    authorized active stream, outside OMI-Port v0
```

Only `PortScope`, `PortTensor`, `PortRoute`, and `PortBinding` exist in OMI-Port v0.

`PortPipe` is explicitly outside v0 because it implies runtime connection.

Canonical rule:

```
A binding may be declared.
A pipe must be authorized.
```

---

12. Minimal C Shape

A minimal non-authoritative OMI-Port shape may be:

```c
typedef struct {
    const char* uri;
    unsigned prefix_len;
} OMI_PortScope;

typedef struct {
    unsigned char gauge; /* F0..FF */
    OMI_PortScope source;
    OMI_PortScope target;

    unsigned short fs;
    unsigned short gs;
    unsigned short rs;
    unsigned short us;

    unsigned short fs_i;
    unsigned short gs_i;
    unsigned short rs_i;
    unsigned short us_i;

    unsigned int reg;
    unsigned int stack;
    unsigned int car;
    unsigned int cdr;
```

```
    int accepted;
    int validated;
    int receipted;
} OMI_PortRoute;
```

Transform function shape:

```
int omi_port_transform(
    const OMI_PortScope* source,
    const OMI_PortScope* target,
    unsigned char gauge,
    OMI_PortRoute* out
);
```

Required default flags:

```
accepted  = 0
validated = 0
receipted = 0
```

Required behavior:

```
derive candidate route
do not connect
do not transmit
do not mount
do not validate
do not receipt
```

---

13. Platform-Agnostic BusyBox Analogy

OMI-Port may be described as a platform-agnostic declarative adapter set.

But the BusyBox analogy must be bounded.

Correct analogy:

```
BusyBox:
    many executable POSIX-like tools behind one compact interface
```

```
OMI-Port:
    many source-target declaration surfaces behind one non-executing interface
```

Incorrect analogy:

```
OMI-Port is not a universal POSIX.
OMI-Port is not a universal kernel.
OMI-Port is not a universal network stack.
OMI-Port is not a universal filesystem.
```

Canonical wording:

```
OMI-Port is BusyBox-like only at the interface-shape level.
It is not BusyBox-like at the execution-authority level.
```

---

14. Relationship to OMI-Canvas

OMI-Port should not live entirely inside the OMI-Canvas type system.

OMI-Canvas may implement or consume OMI-Port declarations, but OMI-Port should exist as a lower adapter boundary.

Correct layering:

```
OMI-Port
    declares source-target route candidates

OMI-Canvas
    may normalize, visualize, type, or construct from those candidates

OMI-Tetragrammatron
    may validate construction geometry later

Receipt authority
    may accept validated state later
```

Therefore:

```
OMI-Port is below OMI-Canvas.
OMI-Canvas may consume OMI-Port.
OMI-Port must not depend on OMI-Canvas.
```

Canonical rule:

```
The port boundary must be usable without the canvas runtime.
```

---

15. Universal OSI Adapter Boundary

OMI-Port MAY define a universal OSI adapter boundary as long as the adapter is dormant.

Dormant OSI adapter means:

```
it names possible layers
it presents normalized declarations
it exposes candidate route shape
it refuses connection authority
```

It does not:

```
open physical links
bind MAC addresses
route IP packets
open TCP or UDP sockets
negotiate sessions
serialize presentation data
call applications
```

Canonical statement:

```
OMI-Port may face OSI.
OMI-Port does not become OSI.
```

---

16. Required Authority Invariant

All OMI-Port implementations MUST preserve:

```
accepted  = 0
validated = 0
receipted = 0
```

until downstream authority explicitly changes those flags.

Forbidden assumptions:

```
port declared means port connected
scope parsed means route valid
gauge selected means route accepted
URI present means resource exists
CIDR suffix means permission exists
adapter shape means projection exists
candidate route means receipt exists
```

Canonical invariant:

```
Declaration is not authority.
Connection requires authority.
Validation and receipt accept.
```

---

17. Canonical Recommendation

OMI-Port should be standardized as:

```
a universal declarative port interface
```

not as:

```
a universal filesystem
a universal network system
a universal OS
a runtime transport
```

The standard should define:

```
URI-CIDR scope grammar
F* gauge transform selector
```

```
FS/GS/RS/US route output
OMI-IMO folded and unfolded notation
non-authoritative candidate flags
adapter handoff shape
```

The standard should leave project-dependent:

```
real filesystem behavior
real network behavior
real canvas behavior
real hardware behavior
real validation
real receipt
real permissioning
```

---

18. One-Line Canon

```
OMI-Port is a dynamic F*-gauged source-target transform that derives canonical
FS/GS/RS/US OMI-IMO place-value route candidates without connecting, validating,
accepting, projecting, or receipting them.
```

---

19. Final Doctrine

OMI-Port standardizes the dormant shape of a port.

It uses URI-CIDR to name and scope source and target surfaces.

It uses the F* Gauge Family to select the dynamic interpolation dialect.

It emits FS/GS/RS/US as canonical four-scope place-value topology.

It renders the route as folded `omi---imo` or unfolded `o---o/---/?---?@---@`.

It may face filesystems, networks, serial devices, LoRa radios, DOM nodes, canvas objects, documents, packets, and sensor events.

It does not connect to them.

It does not mount them.

It does not transmit through them.

It does not validate them.

It does not receipt them.

It declares a route candidate.

Only downstream validation and receipt authority may accept.